

DD2459 Software Reliability 2024
Take-home Exam
Tuesday 5 March, 15.00 – Thursday 7 March, 12.00 midday
(Version 1.0)

Instructions.

Please read the following instructions carefully.

- (1) Only hard-copy exam submissions are accepted. Clearly mark at the top of each sheet of paper you use: (a) your name, (b) the page number.
- (2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)
- (3) There are two ways to **submit** your work. (a) Your work may be **handed in personally by you only** to EECS studentexpedition, E building, level 4, Lindstedsvägen 3, no later than Thursday 7th March 2024 at 12.00 midday. After this time, it will be marked as late, and marks will be subtracted. **Please take identification with you if you submit this way, e.g. a legitimization or passport.** (b) If you are unable to reach the studentexpedition yourself (for example if you are at work) you may **post** your manuscript to: Karl Meinke, EECS School, KTH Kungliga Tekniska Högskolan, 100 44 Stockholm, Sweden. The date on the postmark will be taken as the date of your submission. The deadline of March 7th, 2024 applies to all manuscripts submitted by post.
- (4) If manuscripts are submitted in any other place or by any other means than those described in Part (3) then the examiner and EECS School cannot be held responsible in the case that manuscripts are lost. In the case of postal submission, it is strongly recommended that you keep a digital copy or photocopy in case of postal loss. KTH cannot be held responsible for loss in any national postal system.
- (5) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm, and be aware that I may be in a meeting! You can also e-mail the examiner at karlm@kth.se.
- (6) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates that will have new version numbers 1.x.
- (7) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page. (d) In the case that cheating is suspected, disciplinary action will be taken.
- (8) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

NOTE: For a full score of 100 points you should answer all 4 questions below.

Question 1. (Total 35 points)

Write appropriate *pre-* and *post-conditions* for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).

You should try to avoid both over and under-specification, i.e. write all and only the constraints necessary for each pre- and post condition. Do not omit relevant constraints or add irrelevant ones.

If you are unsure how any mathematical operation/method is defined you may look it up online or in a book.

You do not have to provide Java code for any of the operations, only JML.

Recall that a *finite directed graph* $G = (V, E \subseteq V \times V)$ over a node set $V = \{v_1, \dots, v_n\}$ can be represented on a computer by a 2-dimensional square $n \times n$ Boolean array that encodes the *adjacency matrix*

`boolean [][] Adj`

where

for all $1 \leq i, j \leq n : \text{Adj}[i][j] == \text{true}$ if, and only if $(v_i, v_j) \in E$.

A *clique* C in G is a subset $C \subseteq V$ such any two distinct nodes $v, w \in C$ are connected by an edge $(v, w) \in E$.

- (i) **(10 points)** The *clique property on an undirected graph G*

`boolean clique(boolean [][] Adj, int [] nodeList)`

Input: An adjacency matrix `Adj` for an undirected graph G and a list `nodeList` of nodes occurring in G .

Output: `true` if and only if the set of nodes occurring in `nodeList` forms a clique in G .

- (ii) **(10 points)** A *maximal clique in an undirected graph G*

`int [] maximalClique(boolean [][] Adj)`

Input: An adjacency matrix `Adj` for an undirected graph G .

Output: A maximal sized clique in G (represented as a `nodeList`), i.e. a clique that is not contained in any larger clique in G .

You should reuse your answer to Part (i) to make the JML in your answer shorter.

- (iii) **(10 points)** Write out an algorithm (i.e. pseudo-code) for a test script which uses your JML answer to Part (i) to generate and execute one random test case

(A, C)

for the method

`boolean clique(boolean [][] Adj, int [] nodeList)`

consisting of an undirected adjacency matrix A and a nodeList C in A . Be careful to show clearly how the postcondition of Part (i) is translated into a test oracle to return a verdict in your script.

Finally, carefully analyse whether a randomly chosen test case (A, C) generated by your script is more likely to be a test of a valid clique or an invalid clique for the SUT `clique()`.

- (iv) **(5 points)** Suppose you implement `clique()` and it successfully passes your test script in Part (iii). Is it safe to use your implementation on an adjacency matrix `Adj` for a general directed graph that may not be undirected? Explain your answer.

Question 2. (Total 10 points)

- (i) **(5 points)** Recall that sets of integers can be represented as lists of integers. Write down three different metamorphic equations that should be satisfied by a test for equality on integer sets:

```
boolean SetEquality(List<int> firstSet, List<int>
secondSet)
```

with

Input: Two lists `firstSet`, `secondSet` of `int`.

Output: `true` if the lists `firstSet` and `secondSet` are equal as sets otherwise `false`.

Clearly define the data transformation

$$T: \text{List<int>} * \text{List<int>} \rightarrow \text{List<int>} * \text{List<int>}$$

used on the input pair for each of your three different equations.

- (ii) **(5 points)** For each of your three metamorphic equations e_i , $i = 1, 2, 3$, derived for Part (i), consider its associated transformation T_i . Starting from one seed test case

$$(x_0, y_0) : \text{List<int>} * \text{List<int>}$$

discuss whether the sequence of test cases

$$(x_{j+1}, y_{j+1}) = T_i(x_j, y_j) : j = 0, 1, 2, \dots$$

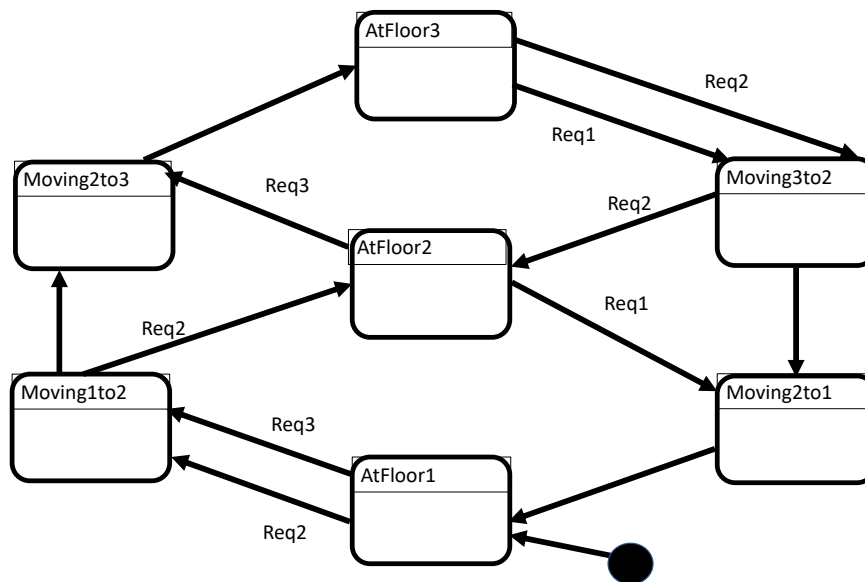
generated by iterating T_i repeats itself.

If all of your test case sequences (x_i, y_i) above are finite, does there exist a metamorphic equation e for `SetEquality` with a transformer T which can generate a test suite of $TS = \{ (x_0, y_0), \dots, (x_k, y_k) = T^k(x_0, y_0) \}$ of unlimited size by simply iterating the transformer T k -times?

Question 3. (Total 35 points)

A building has 3 floors which are serviced by an elevator connecting all floors. An elevator passenger must request that the elevator car comes to her/his floor by means of a request button next to the elevator door on each floor. The three request buttons on floors 1, 2 and 3, when pressed, generate the system events *Req1*, *Req2* and *Req3* respectively. **Note: to simplify this question we will ignore any buttons inside the elevator itself.**

Michael Modelmad drew the following UML Statechart, as a model to design and test the elevator software.



- (i) **(2 points)** Define all 7 different trap properties, using LTL, that would give 100% node coverage of Michael's model.
- (ii) **(3 points)** Define the smallest number of LTL node trap properties that would also give 100% node coverage of Michael's model.
- (iii) **(6 points)** Describe using natural language (English or Swedish) the requirements for correct functional behaviour of the elevator from the perspective of an end user standing on floor 1, floor 2 and floor 3 respectively.
- (iv) **(6 points)** Model your 3 answers to part (iii) above as 3 separate LTL formulas. Each formula should precisely capture the meaning of the corresponding natural language requirement. **Hint:** you may use a *State* variable in your LTL formulas.
- (v) **(6 points)** Michael's sister Mary said that the Michael's model above was wrong. She was able to define three counter-examples, (i.e. execution sequences of the

model) which demonstrated that Michael's model violates all of its three functional behaviour requirements. Write down three counterexamples that Mary could have defined.

- (vi) **(2 points)** Comment on whether your counterexamples in part (v) could be used as test cases for an implementation of the elevator software. Is it possible to test the elevator software for correctness? Motivate your answer.
- (vii) **(10 points)** By introducing just 4 new Boolean variables, **but no new states**, show how you can correct Michael's model to a functionally correct UML Statechart model of an elevator with respect to the user requirements of Part (iv). Use your execution sequences from Part (v) to check if your corrected design fails any Part (iv) requirement. **Hint:** you may change existing transitions and/or introduce new transitions as well as changing conditions and adding assignments for the 4 variables you introduce.

Question 4. (Total 20 points)

- (i) **(6 points)** Recall that a test case tc for a condensation graph G of a program P *satisfies* a control flow test requirement $p = v_1, \dots, v_n$ on G if and only if executing P on tc causes P to take the path p some time during its execution.

We can define the relationship "*implies*" between any two control flow test requirements $reqA$ and $reqB$ on a condensation graph G , by $reqA$ implies $reqB$ if and only if for every test case tc for G ,

if tc satisfies $reqA$ on G then tc satisfies $reqB$ on G .

If $reqA$ implies $reqB$ then in symbols we will write $reqA \Rightarrow reqB$, and if both $reqA \Rightarrow reqB$ and $reqB \Rightarrow reqA$ then we will write $reqA \Leftrightarrow reqB$.

Give rigorous and sound arguments (i.e. proofs or counterexamples) to show (a), (b) and (c) below:

- (a) If $reqA \Rightarrow reqB$ and $reqB \Rightarrow reqC$ then $reqA \Rightarrow reqC$
- (b) If $reqA \Rightarrow reqB$ it does not necessarily follow that $reqB \Rightarrow reqA$.
- (c) If $reqA \Leftrightarrow reqB$ then tc satisfies $reqA$ if and only if tc satisfies $reqB$ and hence $reqA$ and $reqB$ have exactly the same satisfying test cases.

- (ii) **(total 14 points)** Harry Hopeful wanted to unit test the following code

```

1. myTest(int n): boolean
2. {
3.     int x, y, z, s, r;
4.     x:= 1;
```

```

5.      y:= 1;
6.      z:= 1;
7.      repeat {
8.          s:= y+z;
9.          x:= x+1;
10.         y:= z;
11.         z:= x*z;
12.         if (s mod 2 == 1) then
13.             r:=true
14.         else
15.             r:= false;
16.     } until (x-1 > n);
17.     return r;
18. }

```

(a) (2 points) Draw a condensation graph for Harry's code.

(b) (2 points) Harry wrote the following JML precondition for his code
`requires n >= 0`

Write down the correct JML postcondition for this precondition, based on an analysis of the behaviour of Harry's code.

(c) (10 points) Harry thought that the code on line 13 was dead code so that he could never achieve 100% line coverage in testing. Using the method of symbolic evaluation write a constraint table which shows that Harry was wrong. Write out the test case derived from your constraint table and use your postcondition from part (b) to predict your test case output.